Inventory Manager

a project by Elisa Arbaiza and Lily Hillman

Problem Statement

Managing inventory is one of the key essentials for a business of any size, but it becomes not only more essential, but more difficult as a business scales up. Inventory Manager is an attempt at reducing the stress of this task, and will allow store managers and warehouse managers to place orders, check current inventory, and account for shrink via theft or other means. Regular employees will be able to login to view current stock, and see when shipments are expected to arrive from the warehouse to assist customers.

Project Objective

Inventory Manager will not be complete until it is functioning at scale with low latency pre-determined views for use by managers and standard employees alike. This will include proper RBAC, indexing, and backup and restore options to cover common mistakes.

Scope

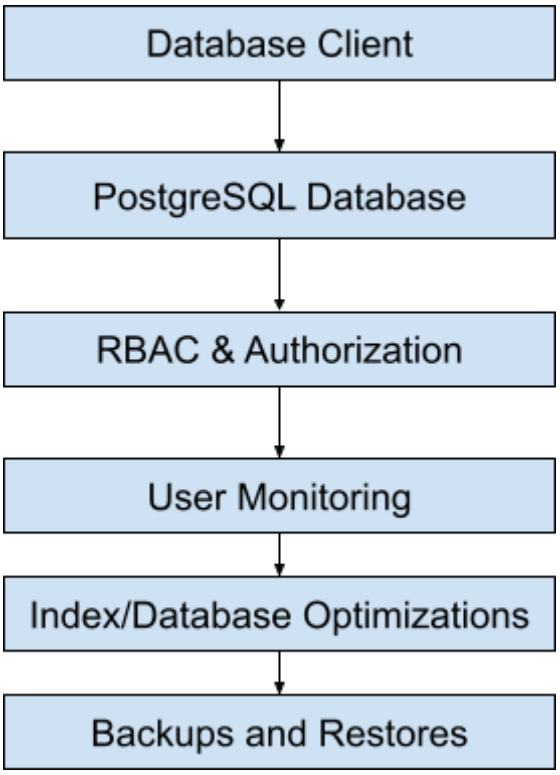
The scope for inventory manager will reflect the needs of an average brick and mortar business model, with tables made for inventory order management, shrinkage, and current inventory. It will also include sections for in-store stocking vs. on-the-shelf stock.

Architecture / Architecture Diagram

Inventory Manager will operate on Version 18 of PostgreSQL.

Architecture Diagram:
Database Client -> PostgreSQL Database -> RBAC & Authorization -> User Monitoring -> Index/Database Optimizations -> Backups and Restores

- Visual of Architecture Diagram text description:



DBA Concepts Applied

Inventory Manager will be created with proper security protocols such as Role-Based Access Control, user logins, and the principle of least privilege. It will use indexes where deemed appropriate by DBA standards, and using the Normal Forms where they belong. The database will regularly backup, with the option to restore these backups in the case of failure events. It will also include cascades where applicable for relevant foreign keys.

Implementation Plan

Each store and warehouse will have its own table with relevant inventory with departmental codes for each item, and inventory counts. All employees will have the ability to check current stock of all items, and the stock of nearby stores. Store managers will be able to edit stock, and add stock requests from their assigned warehouse to be filled. Stock orders will have their own table with estimated delivery dates, stock amounts, order number, and date of order. Roles will be granted by employee authority. For larger tables, indexes will be used on high selectivity columns for faster lookups (e.g. an employee checks for a specific brand of action figure for a customer). The database will automatically backup on intervals, and maintain logs of any

interactions that happened after that backup. Tests will be performed on the database to ensure that large tables do not unnecessarily slow down performance.

Demonstration Plan

We will show the following tests in the video demonstration:

- **Security Test** - We will use RBAC.
- **Show backup/restore workflow** - We will show a simulation of a mistake, and restore the clean backup file to recover the lost data.
- **Demo of before/ after EXPLAIN ANALYZE index comparison.**
- **Show the audit log** - this will show when a store manager edits the stock (will show the audit_log table).

Github Plan

We will use software best practices, branches, and pull requests as part of our normal continued operation on Inventory Manager.

- We will have a README file for how to run the SQL scripts in order.
- SQL files will be included in GitHub.

Team Contributions

Lily Hillman: Writing the Project Proposal
Elisa Arbaiza: Editing and support for the Project Proposal

Member	**Implementation**	**Report**	**Video/Demo**
Lily Hillman	Schema + RBAC Module + Indexing	Architecture and Security, Performance methodology	Security Test via RBAC
Elisa Arbaiza	Performance Tests, Backup/restore	Performance methodology and results. Recovery and testing sections	Demo of before/ after EXPLAIN ANALYZE index comparison. Show backup/restore workflow. Show the audit log

Timeline

As the project length is three weeks, we will break down our project into three separate Sprints.

Sprint 1: (Week 1)
Initialize database, roles, ER diagram, and schemas pertinent to the project.

Sprint 2: (Week 2/3)
Re-evaluate sprint objectives.
Create seeding data.
Implement index and composite indexes for relevant tables and columns.

Sprint 3: Week (3/4)
Test query speed and hardware utilization and improve indexing.
Lay out views for common queries to improve optimization.
Finalize security testing.

Risks/Dependencies

- **Deadline of Project**: We will only have one month to complete the project. We will work on keeping the scope focused more on the PostgreSQL scripts.
- **Software Dependencies**: If one of us uses a different PostgreSQL version, there could be slight differences in syntax. To ensure this does not occur, we will use PostgreSQL 18 and test the scripts with the GitHub repository.
- **Data Requirements:** PostgreSQL will ignore our indexes and use a sequential scan if our tables only have 20 rows. To solve this, we will use the PostgreSQL built-in generate_series() function to help us generate thousands of rows to complete an index performance.

Scope Quality Checklist

Inventory Manager checks off all the required points for the Scope.

- ☑ ~~• The project is large enough to demonstrate meaningful database administration work but small enough to complete during the semester.~~
- ☑ ~~• The project includes implementation and testing, not only research or documentation.~~
- ☑ ~~• The project includes a realistic demonstration scenario.~~
- ☑ ~~• The proposed data and tools are available to the team.~~
- ☑ ~~• The responsibilities are distributed so every member contributes technically.~~
- ☑ ~~• The proposal identifies how results will be measured or verified.~~

Summary

A two-person group will piece together a PostgreSQL warehousing tool to assist employees, managers, and warehouse workers alike to improve their working performance. This tool will allow users to check order status, place orders, fill orders, store stock, warehouse stock, help audit product availability in the store. It will do so in a manner that works efficiently and with low latency, assisted with indexes and pre-determined views. Lily H. will work on the Schemas, Key allocation, ER diagram, Role-Based-Access-Control, and Indexing. Elisa Arbaiza will work on the performance methodology and results, and the recovery and testing sections. All members will contribute to the report and recorded demonstration.

References

18 Concepts for Database Administrators - Oracle.com
SQL Server Security Best Practices - Microsoft.com